**FIAP — Faculdade de Informática e Administração Paulista**
DevOps e Arquitetura Cloud

# ToggleMaster: Infraestrutura como Código, DevSecOps e GitOps na AWS

Relatório de Entrega — Tech Challenge Fase 3

Karlos Rodrigues — RM 374567 — Discord: karlos.rodrigues

São Paulo — 2026

## Sumário

1. Introdução e Contexto do Projeto ..... 3
2. Arquitetura da Solução ..... 4
3. Infraestrutura como Código com Terraform ..... 5
4. Pipeline de CI e DevSecOps ..... 8
5. GitOps com ArgoCD ..... 10
6. Segredos e Segurança ..... 12
7. Validação do Ambiente ..... 12
8. Desafios e Decisões Técnicas ..... 12
9. Considerações Finais e Links do Projeto ..... 14
Apêndice A — Evidências de Execução ..... 15
Apêndice B — De-para: Requisitos do Enunciado × Entregas ..... 17

# 1. Introdução e Contexto do Projeto

Na Fase 2, levei o ToggleMaster para Kubernetes: cinco microsserviços conteinerizados num cluster EKS, com RDS, ElastiCache, SQS e DynamoDB por trás. A arquitetura foi aprovada, mas o enunciado da Fase 3 descreve o preço que se paga por operar tudo à mão: os desenvolvedores rodam `kubectl apply` das próprias máquinas e geram conflitos de versão; as credenciais do banco circulam em arquivos de texto; uma vulnerabilidade numa biblioteca Go passou despercebida e foi para produção; e recriar o ambiente de homologação leva dias porque foi montado no console.

A ordem da DevOps Solutions Inc. — o cenário que o enunciado propõe — é "se não está no código, não existe". Meu trabalho nesta fase foi trocar cada uma dessas quatro dores por uma prática automatizada: a infraestrutura passou a ser **Terraform**, cada serviço ganhou um **pipeline de CI com varreduras de segurança** que bloqueia vulnerabilidade crítica, o deploy deixou de ser um comando executado por pessoas e virou **GitOps com ArgoCD**, e as credenciais saíram dos arquivos para o **AWS Secrets Manager**.

> Segui a Opção B do enunciado — conta pessoal AWS. Por isso o Terraform cria as próprias roles de IAM (cluster, nós, IRSA e a role do GitHub Actions) em vez de depender da `LabRole` do AWS Academy.

## 1.1 Ambiente de desenvolvimento e o que muda entre a minha VM e a AWS

Para não misturar o trabalho com a minha estação pessoal, desenvolvi tudo na mesma VM isolada da Fase 2 (`forge`, Ubuntu 24.04 sobre KVM), onde instalei Terraform, Trivy, GitHub CLI, Go, golangci-lint, gosec, flake8 e bandit. O processo é o mesmo em qualquer destino: escrevo os módulos e rodo `init`, `fmt`, `validate`, `plan` e `apply`. O que muda é só o provider e o backend, não o código.

Eu poderia ter criado toda a infraestrutura na AWS desde o primeiro dia, mas ela é cara para ficar ligada (estimativa na Seção 3.6). Por isso mantive o ambiente desligado durante o desenvolvimento e o subi apenas para validar e gravar. Mas "o processo é o mesmo" não quer dizer que o resultado tenha o mesmo valor, e a Tabela 1 separa o que dá para provar fora da AWS do que não dá.

| Etapa | Na VM, sem criar recursos | Na AWS |
|---|---|---|
| `init`, `fmt`, `validate` | idêntico | idêntico |
| `plan` | exige credencial, mas não cria nada e não gera custo | mostra o que seria criado com as regras reais da conta |
| `apply` da rede (VPC, sub-redes, IGW, route tables) | sem custo na AWS, esses recursos não são cobrados | idêntico |
| `apply` de EKS, RDS e ElastiCache | não dá para reproduzir: o emulador LocalStack só cobre EKS, RDS, ElastiCache e ECR nos planos pagos | cobrado por hora; subo, valido, gravo e destruo |

Tabela 1 — O que o fluxo de trabalho no Terraform prova fora da AWS e o que só a AWS real prova (fonte: docs.localstack.cloud/aws/licensing).

Resumindo: o fluxo e o código são os mesmos, mas um emulador não substitui a prova de que a infraestrutura real sobe — IAM, cotas e tempo de criação do EKS e do RDS só aparecem na AWS. Por isso a evidência desta entrega vem de um `apply` real, de curta duração, seguido de `destroy`.

## 2. Arquitetura da Solução

Organizei a solução em quatro camadas, cada uma versionada em Git: o **Terraform** cria a infraestrutura; o **GitHub Actions** constrói, varre e publica as imagens; o **repositório GitOps** guarda o estado desejado do cluster; e o **ArgoCD**, dentro do EKS, reconcilia o cluster com esse repositório. Nenhuma pessoa nem pipeline de CI executa `kubectl` contra o cluster.

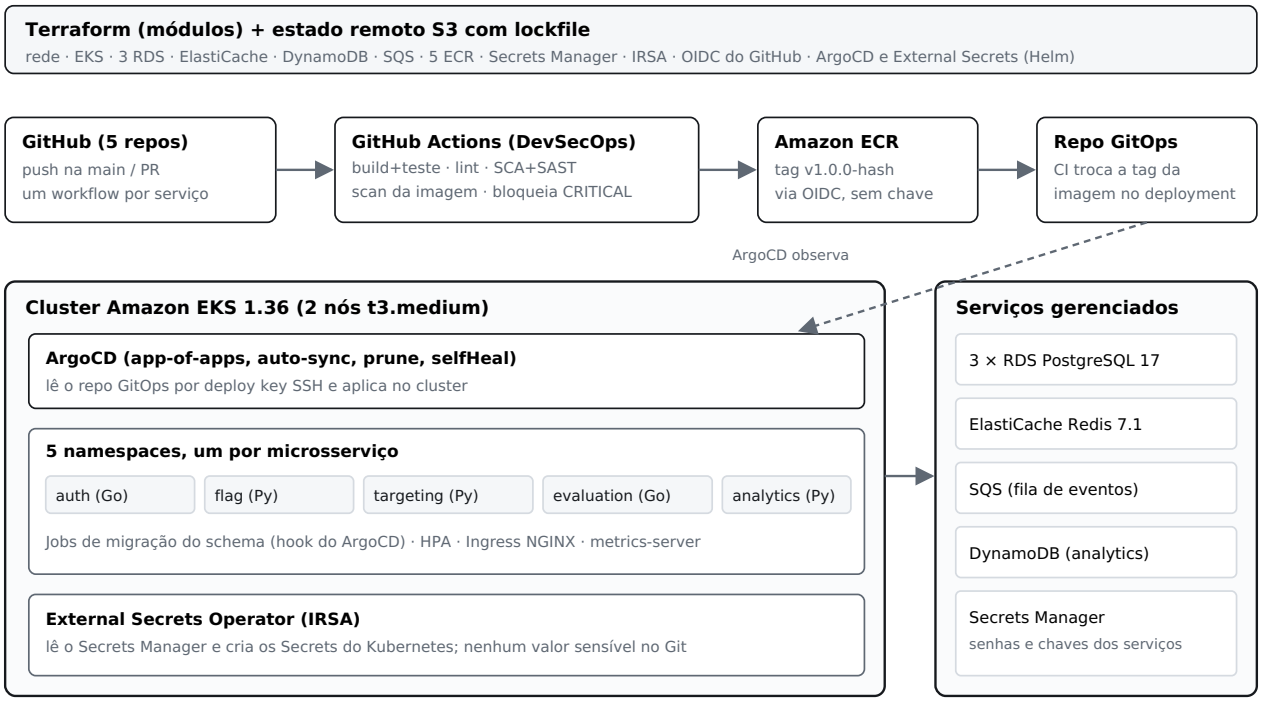



Figura 1 — Visão geral da solução: do commit no GitHub ao Pod no EKS, passando pelo CI de segurança, pelo ECR e pelo repositório GitOps.

| Camada | Ferramenta | Onde está |
|---|---|---|
| Infraestrutura | Terraform 1.17, estado em S3 com lockfile | repositório `togglemaster-fase3` |
| Integração contínua | GitHub Actions, Trivy, gosec, bandit, golangci-lint, flake8 | um `ci.yml` em cada repositório de serviço |
| Estado desejado do cluster | manifestos Kubernetes e Applications do ArgoCD | repositório `togglemaster-gitops` |
| Entrega contínua | ArgoCD 3.5 com sincronização automática | namespace `argocd` no EKS |
| Segredos | AWS Secrets Manager + External Secrets Operator | criados pelo Terraform, sem valores no Git |

Tabela 2 — Camadas da solução e onde cada uma vive.

### 2.1 O caminho de uma mudança

1. Faço um commit na `main` de um serviço (ou abro um Pull Request).
2. O workflow compila, testa, roda o lint e as duas varreduras de segurança; qualquer falha interrompe o fluxo.
3. Passando, o pipeline constrói a imagem, a varre de novo com o Trivy e, só na `main`, a envia ao ECR com a tag `v1.0.0-<hash do commit>`.
4. O último job clona o repositório GitOps, troca a tag no `deployment.yaml` do serviço e faz o commit.
5. O ArgoCD, que observa esse repositório, detecta o commit e sincroniza o cluster sozinho.

## 3. Infraestrutura como Código com Terraform

Substituí a criação via AWS CLI da Fase 2 por um projeto Terraform organizado em módulos, com dois ambientes raiz: `envs/prod` (infraestrutura) e `envs/gitops` (ArgoCD e External Secrets, instalados por Helm depois que o cluster existe).

## 3.1 Módulos

| Módulo | O que cria |
|---|---|
| `rede` | VPC, 2 sub-redes públicas e 2 privadas em 2 zonas, Internet Gateway, NAT opcional e route tables, com as tags que o EKS exige para os Load Balancers |
| `eks` | cluster 1.36 com criptografia de secrets por KMS e logs do control plane, node group `t3.medium` (mín. 1, desejado 2, máx. 4), roles do cluster e dos nós e o provedor OIDC para IRSA |
| `rds` | 3 instâncias PostgreSQL 17 ( `auth` , `flag` , `targeting` ) em sub-rede privada, acessíveis apenas pelo cluster, com backup de 7 dias e autenticação IAM |
| `elasticache` | Redis 7.1 em `cache.t3.micro` , acessível apenas pelo cluster |
| `dynamodb` / `sqs` | tabela `ToggleMasterAnalytics` (chave `event_id` , sob demanda, com PITR) e a fila `togglemaster-evaluation-events` |
| `ecr` | 5 repositórios com tags imutáveis, scan no push e retenção das 20 imagens mais recentes |
| `irsa` | roles do `evaluation-sa` (envia ao SQS) e do `analytics-sa` (consome o SQS e grava no DynamoDB), com permissão só nos recursos necessários |
| `github_oidc` | provedor OIDC do GitHub e a role que o CI assume para publicar no ECR, restrita à branch `main` dos 5 repositórios |
| `segredos` / `eso` | segredos por serviço no Secrets Manager e a role (IRSA) que o External Secrets usa para lê-los |

Tabela 3 — Módulos do projeto Terraform.

## 3.2 Estado remoto

O `terraform.tfstate` não fica local. Criei um bucket S3 com versionamento, criptografia AES256 e acesso público bloqueado, e configurei o backend com `use_lockfile = true`, que dispensa a tabela do DynamoDB para o bloqueio. O nome do bucket fica fora do código, num arquivo `backend.hcl` ignorado pelo Git; o repositório só traz um `backend.hcl.exemplo` .

## 3.3 Versões escolhidas com base nas fontes

Na Fase 2 usei EKS 1.31 e PostgreSQL 16.4. Antes de reaproveitar, consultei a documentação oficial da AWS e as duas versões estavam defasadas: o EKS 1.31 já estava em suporte estendido (US$ 0,60 por hora de cluster, contra US$ 0,10 do suporte padrão) e o 1.34 sai do suporte padrão em 02/12/2026. Usei o **EKS 1.36**, em suporte padrão até agosto de 2027. No RDS, o 16.4 não aparece mais entre as versões menores do calendário de lançamentos, e usei o **PostgreSQL 17**, em suporte padrão até fevereiro de 2030.

### 3.4 Resultado do apply

| Ambiente | Recursos | Duração | Resultado |
|---|---|---|---|
| `envs/prod` | 71 criados, 0 alterados, 0 destruídos | 12 min (20:42 às 20:54 UTC de 02/11/2026) | **Apply complete** |
| `envs/gitops` | 3 criados (External Secrets, ArgoCD, aplicação raiz) | cerca de 2 min | **Apply complete** |

Tabela 4 — Execução real do `terraform apply` na AWS (log completo no Apêndice A).

### 3.5 Segurança do próprio código Terraform

Passei o `trivy config` no projeto antes do primeiro apply. Ele apontou achados que eu mesmo tinha introduzido: sub-redes públicas atribuindo IP público a tudo, endpoint do EKS aberto para `0.0.0.0/0` (CRITICAL), secrets do cluster sem criptografia (HIGH), além de logs do control plane, backups e autenticação IAM do RDS desligados. Corrigi tudo isso. Restaram três exceções conscientes, registradas com justificativa no `.trivyignore`: o endpoint público do EKS (necessário para eu acessar o cluster, mas restrito ao IP da VM), a proteção contra exclusão do RDS (impediria o `destroy` de um ambiente efêmero) e os VPC Flow Logs (custo contínuo, não exigido).

### 3.6 Estimativa de custos

Calculei a estimativa com os preços da API oficial da AWS (`pricing:GetProducts`, região us-east-1) e das páginas de preço do EKS, da VPC e do KMS, consultados em 02/11/2026. A contagem de IPv4 públicos (um do NAT e dois do Load Balancer) e dos discos dos nós (2 volumes gp3 de 20 GB) veio do próprio ambiente criado.

| Componente | Qtd. | Preço unitário | US$ por hora |
|---|---|---|---|
| EKS — plano de controle | 1 | 0,1000 /h | 0,1000 |
| EC2 t3.medium Linux (nós) | 2 | 0,0416 /h | 0,0832 |
| RDS PostgreSQL db.t3.micro, Single-AZ | 3 | 0,018 /h | 0,0540 |
| ElastiCache cache.t3.micro, Redis | 1 | 0,017 /h | 0,0170 |
| NAT Gateway (mais US$ 0,045 por GB processado) | 1 | 0,045 /h | 0,0450 |
| Classic Load Balancer do Ingress | 1 | 0,025 /h | 0,0250 |
| IPv4 público em uso (1 NAT + 2 Load Balancer) | 3 | 0,005 /h | 0,0150 |
| **Total por hora** | | | **0,3392** |
| RDS — armazenamento gp2 | 60 GB | 0,115 /GB-mês | 6,90 /mês |
| EBS gp3 dos nós | 40 GB | 0,08 /GB-mês | 3,20 /mês |
| Secrets Manager | 5 | 0,40 /mês | 2,00 /mês |
| KMS — chave gerenciada pelo cliente | 1 | 1,00 /mês | 1,00 /mês |
| **Total mensal (730 h ligados + itens mensais)** | | | **260,72** |

Tabela 5 — Estimativa de custo do ambiente completo, em US$, região us-east-1. Não inclui tráfego de dados (o NAT cobra por GB), CloudWatch Logs do control plane nem o armazenamento no ECR; o DynamoDB e o SQS são sob demanda e têm volume desprezível nesta demonstração.

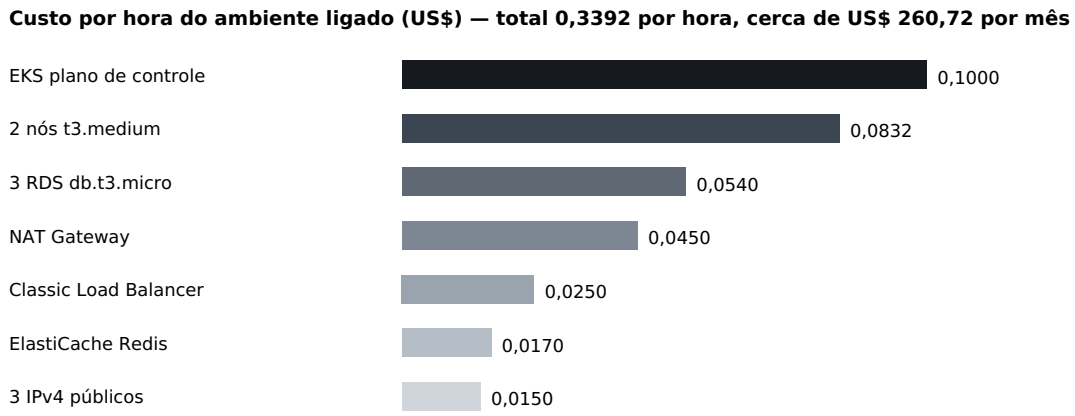



Figura 2 — Estimativa de custos AWS: peso de cada componente no valor por hora (gerada a partir da Tabela 5, com preços da API oficial da AWS).

O número que importa para a decisão é o por hora. Manter o ambiente ligado apenas durante a validação e a gravação, algumas horas, custa em torno de US$ 1 a 3. Deixá-lo ligado um mês inteiro custaria cerca de US$ 261, e por isso o `terraform destroy` faz parte do encerramento desta entrega.

## 4. Pipeline de CI e DevSecOps

Cada um dos cinco serviços tem o seu `ci.yml`, gerado a partir de dois modelos (Go e Python) e validado com o `actionlint`. O workflow roda em todo Pull Request e em todo push na `main`.

| Job | Serviços em Go | Serviços em Python | Regra |
|---|---|---|---|
| Build e testes unitários | `go build`, `go vet`, `go test` | `compileall` e `pytest` se houver testes | falha interrompe o fluxo |
| Linter | golangci-lint | flake8 | falha interrompe o fluxo |
| SCA (dependências) | Trivy em modo `fs` | | **CRITICAL** bloqueia |
| SAST (código-fonte) | gosec | bandit | severidade alta bloqueia |
| Docker build e scan | build, Trivy na imagem e push ao ECR só na `main` | | **CRITICAL** bloqueia |
| Atualização do GitOps | troca a tag no `deployment.yaml` do repositório GitOps | | só na `main` |

Tabela 6 — Estágios do pipeline de cada serviço.

### 4.1 Autenticação na AWS sem chave estática

O CI não guarda nenhuma chave da AWS. O job de imagem assume, por OIDC, uma role do IAM criada pelo Terraform e restrita à branch `main` dos cinco repositórios, com permissão apenas de envio ao ECR. Isso elimina a classe de problema de credencial vazada em segredo

de repositório. O único segredo que o CI guarda é o `GITOPS_TOKEN`, usado para gravar no repositório de manifestos.

### 4.2 Resultado

Os cinco pipelines terminaram verdes e publicaram as imagens no ECR com a tag do commit (por exemplo, `v1.0.0-f0ea038` no `auth-service`). O código recebido da FIAP, porém, não passou no pipeline de primeira, e esse caminho está contado nas Seções 8.1 a 8.4. A Tabela 7 resume o histórico real.

| Quando (UTC) | O que aconteceu | Causa |
|---|---|---|
| 20:52 | 5 pipelines falham sem criar nenhum job | condição `if` sem aspas no workflow; o GitHub rejeitou o arquivo |
| 20:56 | Go falha no job de segurança | `gosec@latest` exige Go 1.26; o `go.mod` pede 1.21 |
| 20:56 | Python falha no login da AWS | a conta do GitHub usa *subject* imutável no OIDC |
| 20:58 | Go falha no scan da imagem | Trivy encontrou **CVE-2025-68121** (CRITICAL) na stdlib do Go 1.21.13 |
| 21:10 | **5 pipelines verdes** | correções das quatro causas acima |

Tabela 7 — Cronologia real das execuções do CI em 02/11/2026.

### 4.3 Demonstração do bloqueio de segurança (Pull Request nº 1)

Para o vídeo fiz o que o enunciado pede: abri um Pull Request no `flag-service` acrescentando `Django==3.0`, uma dependência com vulnerabilidades conhecidas, e deixei o pipeline rodar de verdade. A Tabela 8 resume o que aconteceu.

| Passo | Resultado |
|---|---|
| Commit com `Django==3.0` | o job **Segurança (SCA e SAST)** **falhou**: o Trivy listou 4 vulnerabilidades **CRITICAL** (CVE-2019-19844, CVE-2020-7471, CVE-2021-35042 e CVE-2025-64459); os jobs de imagem e de GitOps ficaram em *skipped* |
| Commit de correção (revert) | os jobs de build, linter e segurança passaram, a imagem foi construída e varrida; sem push ao ECR, porque é um Pull Request |
| Merge na `main` | pipeline completo: imagem `v1.0.0-e238cde` no ECR e commit de tag no repositório GitOps |
| ArgoCD | detectou o commit sozinho e aplicou o rollout; a nova revisão aparece no histórico da Application (id 3, 23:35:51 UTC) |

Tabela 8 — Demonstração real do bloqueio de segurança e do ciclo até o cluster, em 02/11/2026 (a saída completa está no Apêndice A.8).

# 5. GitOps com ArgoCD

## 5.1 Repositório de manifestos

O repositório `togglemaster-gitops` contém apenas YAML: `apps/<serviço>/` com Deployment, Service, ConfigMap, ExternalSecret e, quando existem, HPA e ServiceAccount; `base/` com namespaces, o ClusterSecretStore e o Ingress; e `argocd/` com uma Application por componente. Partiu dos manifestos da Fase 2, com três ajustes: os Secrets com valores viraram ExternalSecrets, o ServiceAccount do IRSA passou a ser declarado no repositório, e as tabelas dos bancos passaram a ser criadas por Jobs de migração (Seção 8.7).

## 5.2 ArgoCD

Instalei o ArgoCD no EKS pelo Terraform (chart Helm `argo-cd` 10.11.6, versão fixada). Uma aplicação raiz (*app-of-apps*) lê a pasta `argocd/` do repositório GitOps e cria as demais Applications, todas com sincronização automática, `prune` e `selfHeal`. Qualquer alteração feita à mão no cluster é revertida; o Git é a única fonte de verdade.

| Application | Origem | Estado |
|---|---|---|
| `auth-service`, `flag-service`, `targeting-service`, `evaluation-service`, `analytics-service` | repositório GitOps, `apps/<serviço>` | **Synced / Healthy** |
| `base` | repositório GitOps, `base/` | **Synced / Healthy** |
| `ingress-nginx`, `metrics-server` | charts Helm oficiais, versões fixadas | **Synced / Healthy** |
| `raiz` | repositório GitOps, `argocd/` | **Synced / Healthy** |

Tabela 9 — Applications gerenciadas pelo ArgoCD (saída real de `kubectl get applications` no Apêndice A).

## 5.3 Atualização automática da tag

Ao fim do pipeline, o job `gitops` clona o repositório de manifestos, troca a tag da imagem no `deployment.yaml` do serviço pelo valor recém-publicado e grava o commit. Os cinco commits resultantes (por exemplo, "auth-service: imagem v1.0.0-f0ea038") estão no histórico do repositório. O ArgoCD detectou cada um e aplicou o rollout sem nenhuma intervenção minha: os Pods antigos, presos em `ErrImagePull` porque a tag provisória `v0.0.0-inicial` não existia no ECR, foram substituídos pelos novos.

## 5.4 Acesso do ArgoCD ao repositório

O ArgoCD lê o repositório por SSH, com uma deploy key somente leitura cadastrada só nesse repositório. Assim a configuração funciona igual com o repositório público ou privado, e fechá-lo no final não exige mudar nada no cluster.

## 5.5 Interface do ArgoCD

As Figuras 3 e 4 são capturas reais da interface do ArgoCD, tiradas depois do deploy da demonstração. A primeira mostra as nove Applications `Synced` e `Healthy`, entre elas os cinco

microsserviços; a segunda mostra a árvore de recursos do `flag-service`, com o ExternalSecret, o Job de migração do schema e o ReplicaSet novo.

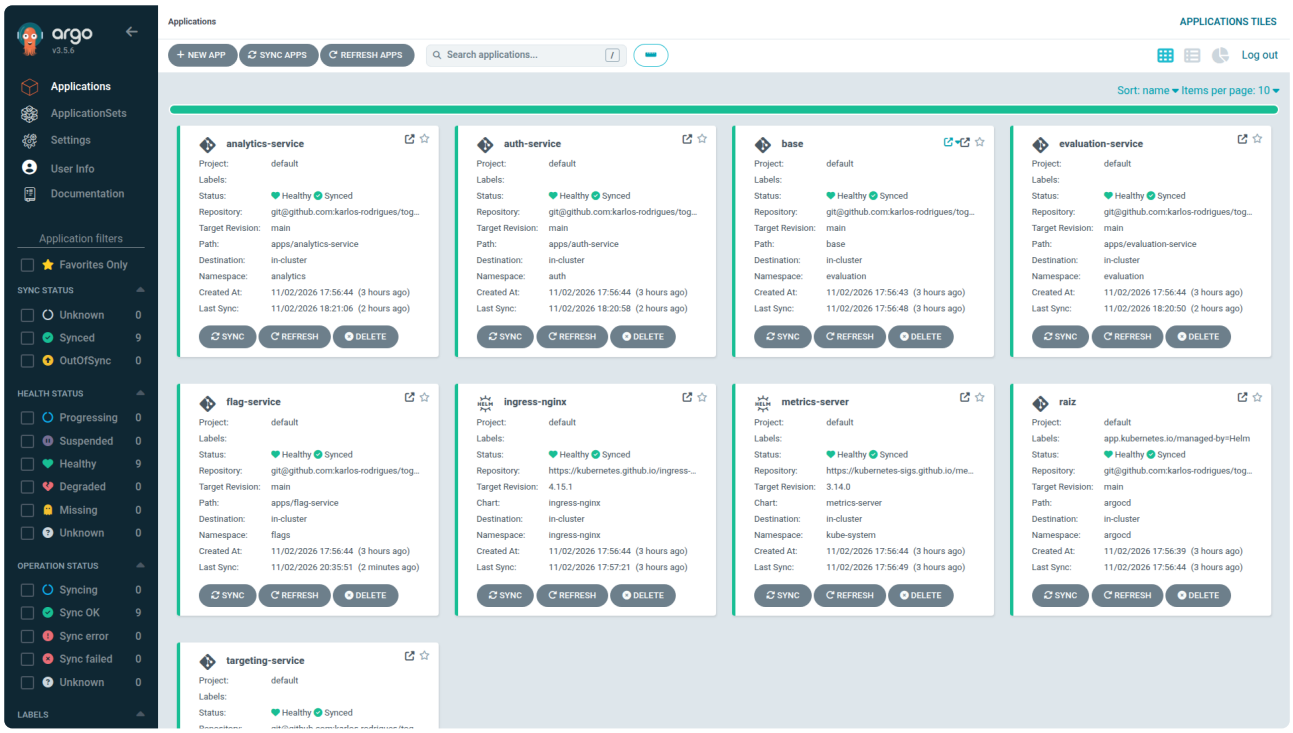



Figura 3 — Interface do ArgoCD: as Applications gerenciadas, todas Synced e Healthy.

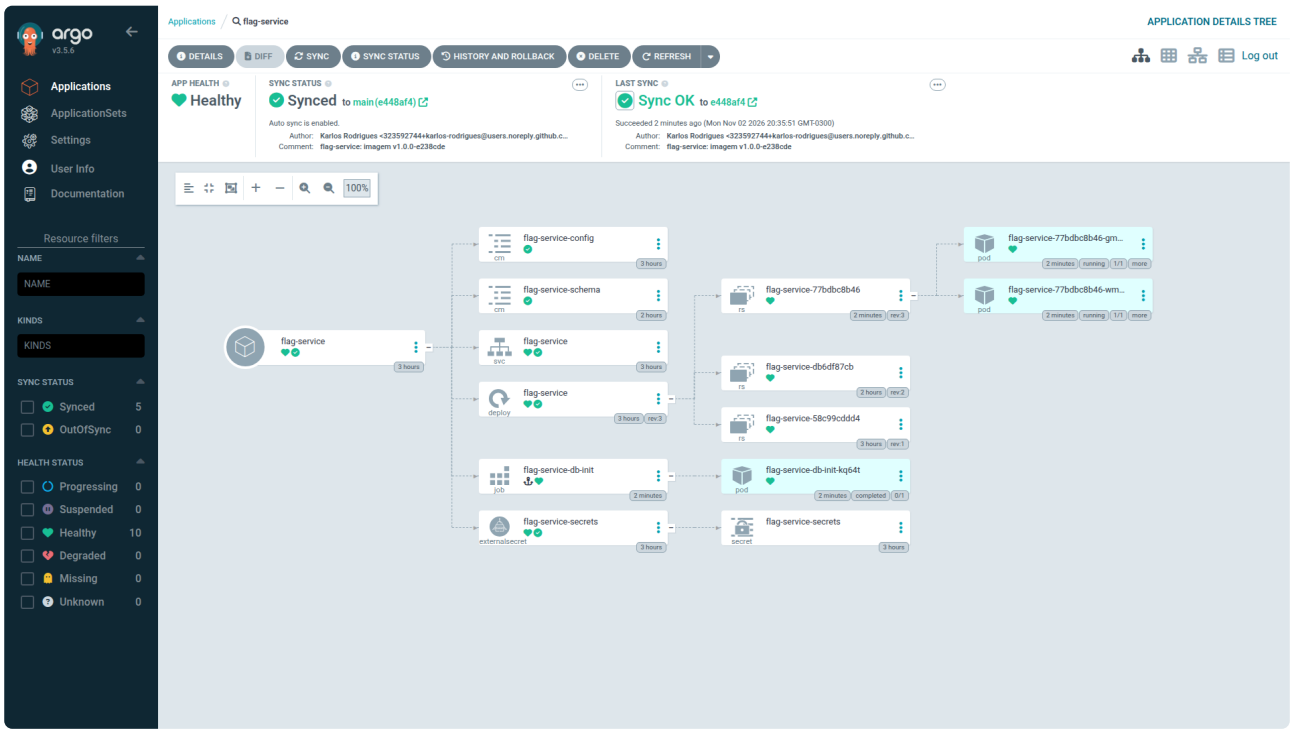



Figura 4 — Interface do ArgoCD: árvore de recursos do flag-service sincronizada a partir do repositório GitOps.

## 6. Segredos e Segurança

O segundo problema do enunciado era a credencial do banco trafegando em arquivos de texto. Na Fase 2, a connection string ia para um `Secret` montado à mão. Nesta fase o caminho passou a ser:

1. o Terraform gera as senhas e a `MASTER_KEY` (`random_password`) e as grava no **AWS Secrets Manager**, um segredo por serviço, já com a connection string pronta;
2. o **External Secrets Operator**, com uma role IRSA que só lê esses cinco segredos, copia o conteúdo para `Secrets` do Kubernetes e o mantém atualizado a cada minuto;
3. o repositório GitOps traz apenas o `ExternalSecret` (o nome do segredo, nunca o valor).

Além disso, o acesso aos bancos e ao Redis é restrito ao Security Group do cluster, o tráfego do CI para a AWS usa OIDC e os repositórios do ECR têm tags imutáveis e scan no push.

## 7. Validação do Ambiente

Com o ambiente no ar, validei o fluxo de ponta a ponta pelo Ingress, chamando as mesmas rotas da Fase 2.

| Verificação | Resultado obtido |
|---|---|
| Cluster | 2 nós `Ready` na versão v1.36.5 |
| ArgoCD | 9 Applications `Synced` e `Healthy` |
| External Secrets | 5 ExternalSecrets em `SecretSynced` |
| Migração do schema | 3 Jobs concluídos (`CREATE TABLE` e trigger em cada banco) |
| Chave de serviço | `POST /auth/admin/keys` devolveu uma chave `tm_key_…` de 71 caracteres |
| Criação da flag e da regra | `enable-new-dashboard` e regra `PERCENTAGE` de 50%, ambas gravadas |
| Avaliação de 8 usuários | 7 resultados `true` e 1 `false`; o resultado é determinístico por `user_id` e `flag_name` |
| Fluxo assíncrono | 8 eventos publicados no SQS e **8 itens** na tabela `ToggleMasterAnalytics` |

Tabela 10 — Validação funcional do ambiente criado pelo Terraform e entregue pelo ArgoCD.

## 8. Desafios e Decisões Técnicas

### 8.1 O código recebido reprovou no meu próprio pipeline

Antes de publicar, rodei as ferramentas na VM e o código dos serviços já não passava. No `auth-service` e no `evaluation-service`, o golangci-lint apontou retornos de erro ignorados e o uso do `io/ioutil`, obsoleto desde o Go 1.19. O gosec marcou como HIGH (SSRF, G704) as chamadas HTTP do `evaluation-service`. Nos três serviços Python, o flake8 trouxe de 17 a 43 avisos de estilo e um import sem uso. Corrigi sem mudar comportamento: `_ =` explícito nos retornos, `io.ReadAll`, autopep8 e autoflake. No SSRF, anotei com `#nosec` e justificativa,

porque a URL vem de variáveis definidas pelo operador no ConfigMap e nunca de entrada do usuário. Registrei tudo em `docs/PATCHES.md`.

### 8.2 O Trivy barrou uma vulnerabilidade real

O scan de imagem reprovou os dois serviços Go com o **CVE-2025-68121** (CRITICAL, validação incorreta de certificado no `crypto/tls`), presente na biblioteca padrão do Go 1.21.13 usada no estágio de build. É exatamente o tipo de falha que o enunciado conta que "passou despercebida e foi para produção". Subi o build para `golang:1.25-alpine` (Go 1.25.14, que contém a correção), validei na VM que as duas imagens passam no Trivy sem nenhum CRITICAL e o pipeline voltou a ficar verde. O CVE só foi encontrado porque o scan roda sobre a imagem final, e não apenas sobre as dependências do `go.mod`.

### 8.3 gosec e a versão do Go

O `gosec@latest` exige Go 1.26 ou superior, mas o `go.mod` dos serviços pede Go 1.21 e o workflow usava essa versão. O job de segurança passou a usar o Go estável só para a análise, o que não afeta a versão com que o serviço é compilado.

### 8.4 Aspas perdidas no workflow

Os cinco pipelines falharam sem executar nenhum job. O `actionlint` mostrou que a condição `github.event_name == 'push'` tinha perdido as aspas ao ser gerada pelo meu shell, e o GitHub rejeitou o arquivo inteiro. Corrigi o modelo, regerei os cinco workflows e passei a validá-los com o `actionlint` antes de cada push.

### 8.5 OIDC do GitHub com subject imutável

A AWS recusava o login do CI (`Not authorized to perform sts:AssumeRoleWithWebIdentity`), embora a role e o provedor estivessem corretos. Consultei a configuração da conta no GitHub e vi que ela usa *subject* imutável: o campo `sub` do token traz os IDs numéricos do dono e do repositório (`repo:karlos-rodrigues@<id>/flag-service@<id>:ref:refs/heads/main`), e a trust policy que eu tinha escrito usava só os nomes. Passei a usar os IDs, lidos pela API do GitHub e declarados em uma variável do Terraform. O `plan` mostrou uma única alteração, na trust policy.

### 8.6 Senha do banco: gerenciada pelo RDS ou gerada pelo Terraform

Comecei com `manage_master_user_password`, em que o RDS guarda a senha no Secrets Manager. O problema é que o ExternalSecret precisaria do nome aleatório desse segredo, que mudaria a cada ambiente e voltaria a colocar informação específica do ambiente no Git. Optei por gerar a senha no Terraform e gravar o segredo já montado. O custo dessa escolha é que a senha passa a existir no *state*, que está num bucket privado, versionado e criptografado. Considero a troca aceitável para o ganho de manter o repositório GitOps livre de valores do ambiente.

### 8.7 As tabelas dos bancos não existiam

Com o cluster no ar, o `auth-service` respondia 500 e o log mostrava `relation "api_keys" does not exist`: na Fase 2 eu criava as tabelas à mão, e isso contraria "se não está no código, não existe". Criei, no repositório GitOps, um Job por serviço com banco (hook *Sync* do ArgoCD,

com a ordem garantida por *sync waves*: ExternalSecret, depois ConfigMap com o `init.sql`, depois o Job). Os três `init.sql` já eram idempotentes, então o Job pode rodar a cada sincronização sem risco.

### 8.8 Dependência circular da chave de serviço

O `evaluation-service` precisa de uma chave de API que só o `auth-service` pode gerar, e o `auth-service` só existe depois do deploy. É o mesmo ciclo da Fase 2. O Terraform grava um valor provisório no segredo e ignora mudanças nele (`ignore_changes`), e um passo único pós-apply, documentado em `docs/BOOTSTRAP-POS-APPLY.md`, gera a chave, atualiza o Secrets Manager e reinicia o deployment. Não automatizei esse passo; é um ponto a melhorar.

### 8.9 Custo e escolha de versões

Descrevi na Seção 3.3 a troca do EKS 1.31 e do PostgreSQL 16.4 por versões em suporte padrão, o que evita pagar suporte estendido. A Seção 3.6 explica por que o ambiente fica ligado apenas durante a validação.

### 8.10 Ingress NGINX descontinuado

Mantive o Ingress NGINX da Fase 2 para não misturar duas mudanças, mas, ao pesquisar a versão do chart, descobri que o projeto foi aposentado em março de 2026 e já não recebe correções de segurança (comunicado do Kubernetes de janeiro de 2026). É uma limitação consciente desta entrega: em um ambiente real, eu migraria para o Gateway API ou para outro controller mantido.

## 9. Considerações Finais e Links do Projeto

Quatro dores do enunciado viraram quatro mecanismos: o ambiente se recria com `terraform apply` (12 minutos na AWS real), nenhuma mudança chega ao cluster sem passar por um pipeline que varre dependências, código e imagem, o deploy é feito pelo ArgoCD a partir do Git, e nenhuma senha está em arquivo ou no repositório. O melhor argumento a favor do desenho veio do próprio pipeline, que barrou um CVE crítico real antes que chegasse ao cluster.

### 9.1 Limitações assumidas

- Os repositórios dos serviços não têm testes unitários; o job de testes compila e verifica, mas não cobre regras de negócio.
- O Ingress NGINX está sem manutenção upstream (Seção 8.10).
- O endpoint público da API do EKS continua ligado, restrito ao IP da VM de trabalho; a alternativa seria VPN ou bastião.
- O SAST bloqueia em severidade alta, porque as ferramentas escolhidas não têm nível crítico.
- O bootstrap da chave de serviço é um passo manual documentado.
- O vídeo não tem narração: as falas aparecem como texto na tela, como no vídeo da demonstração da Fase 2.

### 9.2 Links

| Item | Endereço |
|---|---|
| Código Terraform, workflows-modelo e documentação | `https://github.com/karlos-rodrigues/togglemaster-fase3` |
| Manifestos GitOps | `https://github.com/karlos-rodrigues/togglemaster-gitops` |
| Serviços com o workflow de CI | `https://github.com/karlos-rodrigues/{auth,flag,targeting,evaluation,analytics}-service` |
| Vídeo de demonstração (até 20 min) | `https://github.com/karlos-rodrigues/togglemaster-fase3/blob/main/docs/TC-fase3-video-completo.mp4` (7 min 16 s, 1080p, sem narração; as falas aparecem como legenda na tela) |

Tabela 11 — Repositórios e vídeo da entrega.

## Apêndice A — Evidências de Execução

Saídas reais coletadas no ambiente em 02/11/2026.

### A.1 Terraform

```
# terraform prod: início 2026-11-02T20:42:27Z, fim 2026-11-02T20:54:28Z
Plan: 71 to add, 0 to change, 0 to destroy.
Apply complete! Resources: 71 added, 0 changed, 0 destroyed.

# terraform gitops
helm_release.external_secrets: Creation complete after 1m5s [id=external-secrets]
helm_release.argocd: Creation complete after 1m35s [id=argocd]
helm_release.raiz: Creation complete after 4s [id=togglemaster-raiz]
Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
```

### A.2 Nós do cluster

```
NAME                          STATUS  VERSION              INTERNAL-IP
ip-10-0-10-189.ec2.internal  Ready   v1.36.5-eks-6e7947a  10.0.10.189
ip-10-0-11-89.ec2.internal   Ready   v1.36.5-eks-6e7947a  10.0.11.89
```

### A.3 Applications do ArgoCD

```
NAME                SYNC STATUS   HEALTH STATUS
analytics-service   Synced        Healthy
auth-service        Synced        Healthy
base                Synced        Healthy
evaluation-service  Synced        Healthy
flag-service        Synced        Healthy
ingress-nginx       Synced        Healthy
metrics-server      Synced        Healthy
raiz                Synced        Healthy
targeting-service   Synced        Healthy
```

## A.4 External Secrets

```
NAMESPACE   NAME                       STATUS        READY
analytics   analytics-secrets          SecretSynced  True
auth        auth-secrets               SecretSynced  True
evaluation  evaluation-secrets         SecretSynced  True
flags       flag-service-secrets       SecretSynced  True
targeting   targeting-service-secrets  SecretSynced  True
```

## A.5 Imagens no ECR (tag = hash do commit)

```
auth-service: v1.0.0-f0ea038
flag-service: v1.0.0-e4dcb55
targeting-service: v1.0.0-8f84190
evaluation-service: v1.0.0-bf0203d
analytics-service: v1.0.0-4cb3624
```

## A.6 Commits de tag feitos pelo CI no repositório GitOps

```
b4cef97 Karlos Rodrigues gitops: Jobs de migração do schema (hook do ArgoCD) para auth, flag e
targeting
54584cc Karlos Rodrigues evaluation-service: imagem v1.0.0-bf0203d
f9ab1e1 Karlos Rodrigues auth-service: imagem v1.0.0-f0ea038
baab8be Karlos Rodrigues analytics-service: imagem v1.0.0-4cb3624
66d761a Karlos Rodrigues targeting-service: imagem v1.0.0-8f84190
5d96d1f Karlos Rodrigues flag-service: imagem v1.0.0-e4dcb55
59de0ee Karlos Rodrigues argocd: acesso ao repositório privado por SSH
14c7564 Karlos Rodrigues gitops: manifestos dos 5 serviços, base e Applications do ArgoCD
```

## A.7 Itens gravados pelo analytics-service

```
itens DynamoDB: 8
```

## A.8 Demonstração do bloqueio de segurança (Pull Request nº 1 do flag-service)

```
$ gh api .../actions/runs/41545658425/jobs   # resultado: failure
   success  Linter
   failure  Segurança (SCA e SAST)
   success  Build e testes unitários
   skipped  Docker build, scan e push
   skipped  Atualiza a tag no repositório GitOps
$ gh run view 41545658425 --log-failed | grep CVE   # Trivy, severidade CRITICAL
Total: 4 (CRITICAL: 4)
| Django  | CVE-2019-19844 | CRITICAL | fixed  | 3.0               | 1.11.27, 2.2.9, 3.0.1  |
|         | CVE-2020-7471  |          |        |                   | 1.11.28, 2.2.10, 3.0.3 |
|         | CVE-2021-35042 |          |        |                   | 3.2.5, 3.1.13          |
|         | CVE-2025-64459 |          |        |                   | 5.2.8, 5.1.14, 4.2.26  |
```

## Apêndice B — De-para: Requisitos do Enunciado × Entregas

| Requisito do enunciado | Entrega | Status |
|---|---|---|
| Networking: VPC, sub-redes públicas e privadas, IGW, route tables | módulo `rede` | **Feito** |
| Cluster EKS e Node Groups | módulo `eks` (Opção B: roles criadas pelo Terraform) | **Feito** |
| 3 RDS PostgreSQL, 1 ElastiCache Redis, 1 DynamoDB | módulos `rds`, `elasticache`, `dynamodb` | **Feito** |
| 1 fila SQS e 5 repositórios ECR | módulos `sqs` e `ecr` | **Feito** |
| Estado remoto em S3 (e `use_lockfile`) | backend S3 com lockfile, versionado e criptografado | **Feito** |
| Workflow de CI para cada um dos 5 serviços, em PR e push na main | 5 arquivos `ci.yml` | **Feito** |
| Build e testes unitários | build, vet e test (Go); compileall e pytest (Python) | **Feito** — o enunciado diz "se houver"; os repositórios não têm testes |
| Linter / análise estática | golangci-lint e flake8 | **Feito** |
| SCA e SAST, com bloqueio em CRITICAL | Trivy fs; gosec e bandit | **Feito** — SAST bloqueia em alta severidade |
| Docker build, scan, login no ECR e push com hash do commit | job de imagem com Trivy e OIDC | **Feito** |
| Repositório GitOps separado | `togglemaster-gitops` | **Feito** |
| ArgoCD instalado no EKS | Helm pelo Terraform | **Feito** |
| CI atualiza a tag da imagem no GitOps | job `gitops` do workflow | **Feito** |
| ArgoCD sincroniza automaticamente | auto-sync, prune e selfHeal | **Feito** |
| Interface do ArgoCD gerenciando os 5 serviços | Figuras 3 e 4 e telas no vídeo | **Feito** |
| Vídeo de demonstração (até 20 min) | TC-fase3-video-completo.mp4 (7 min 16 s, sem narração) | **Feito** |
| Código no repositório | Terraform, workflows e manifestos publicados | **Feito** |
| Relatório: participantes, links, desafios e decisões | este documento | **Feito** |
| Print da estimativa de custos AWS | Tabela 5 e Figura 2 | **Feito** |

Tabela 12 — Cobertura dos requisitos do enunciado da Fase 3.